AAS in AI Software Engineering

FE101

For loops

Today

Welcome to your next lesson: For loops.

I'm preparing your personalized lesson based on what you've learned so far, what comes next, and how you can build your skills step by step.

Keep it up, every step counts.

Let’s connect this to what you already know 👍

With a while loop, you’ve been doing this pattern:

Javascript

```
let i = 0;              // 1. start
while (i < 5) {         // 2. check
  console.log(i);       // 3. body
  i = i + 1;            // 4. update
}
```

A classic for loop just puts those 3 pieces (start, check, update) in one line at the top:

Javascript

```
for (let i = 0; i < 5; i = i + 1) {
  console.log(i);
}
```

So it’s the same 4 steps, just packed into the for (...) header.

First tiny step:

In your code editor, type and run this exact for loop:

```
Javascript
for (let i = 0; i < 3; i = i + 1) {
  console.log("i is:", i);
}
```

Run it (you can hit Run first, then Submit code when you're happy), then tell me:

Did the output match what you expected, or did anything surprise you?

```
Javascript
for (let i = 0; i < 3; i = i + 1) {
  console.log("i is:", i);
}
```

Nice, Ron — that loop looks perfect and the output is exactly what I'd expect 🎯

You just saw a classic for loop in action:

`let i = 0` → start value
`i < 3` → condition checked before each run
`i = i + 1` → how i changes after each run

So it logged 0, then 1, then 2.

Quick check-in (one idea at a time):

In this loop, where does the number of times it runs come from — the 0, the 3, or the `i = i + 1` part?

You don't need to explain, just answer with one of these: 0, 3, or `i = i + 1`.

the 3 specifically the check if `i < 3`

Exactly right ✅

The condition `i < 3` is what controls how long the loop runs. Once that condition is false, the loop stops.

Now let's make the index actually useful, not just printed.

Think of this array:

Javascript
let colors = ["red", "green", "blue"];
A classic for loop lets us use the index i to grab each color:

Javascript
let colors = ["red", "green", "blue"];

for (let i = 0; i < colors.length; i = i + 1) {
  console.log(i, colors[i]);
}
Step by step, i will be 0, 1, 2, and we use colors[i].

Your turn in the code editor 🧪

Create the colors array exactly as above.
Add the for loop exactly as above.
Run it, then Submit code.

Don’t tell me the output yet—just say “done” after you submit.

Javascript
let colors = ["red", "green", "blue"];

for (let i = 0; i < colors.length; i = i + 1) {
  console.log(i, colors[i]);
}
Nice work, Ron — that’s a textbook classic for loop using the index 👌

You used colors.length, so it will still work even if the array size changes. Great habit.

So, so far with classic for:

It’s great when you need the index (i) to access array[i].
The 3 key parts live in the header: start (let i = 0), condition (i < colors.length), update (i = i + 1).
Next step, I want to slow down and trace what happens.

Pick one index for this loop and answer:

When i is 1, what will colors[i] be in your code?

green

Exactly 🌱

When i is 1, colors[i] is colors[1], which is "green" — that matches your output.

So you've seen:

i → the index
colors[i] → the value at that index
Now I want to show you a different loop that focuses on values, not indexes: for...of.

It's very similar to Python's for color in colors:.

Before I show it, quick check:

Have you used Python's for loop before? Just answer yes or no.

Yes

Perfect, that helps a lot 🙌

In Python you'd do:

Python
```
for color in colors:
    print(color)
```
In JavaScript, the closest thing is:

Javascript
```
for (const color of colors) {
  console.log(color);
}
```
Here, color is each value, one by one — no index needed.

Next step:

In your code editor, below your existing code (or in a fresh file), add:

Javascript
```
let colors = ["red", "green", "blue"];

for (const color of colors) {
  console.log(color);
}
```
Run it, then Submit code, and just tell me: did it print each color on its own line?

yes but had t remove the duplicate let colors line

Nice catch removing the duplicate let colors 👍 That's exactly what you should do—only define it once.

So now you've seen:

Classic for with an index: for (let i = 0; i < colors.length; i = i + 1)
for...of with values: for (const color of colors)

Quick understanding check (one detail):

In this loop:

Javascript
```
for (const color of colors) {
  console.log(color);
}
```
Is color holding the index or the value from the array?

the value

Exactly 💡

In for (const color of colors), color is each value: "red", "green", "blue" — never 0, 1, 2.

So we now have:

Classic for → good when you need the index.
for...of → good when you only care about values.
Let’s do a tiny tracing exercise with console.log to really see the loop steps.

In your editor, write this and run it:

Javascript
```
let nums = [10, 20, 30];

for (let i = 0; i < nums.length; i = i + 1) {
  console.log("index:", i, "value:", nums[i]);
}
```
Submit the code, then tell me: what 3 lines did it print?

```
index: 0 value: 10
index: 1 value: 20
index: 2 value: 30
```

Perfect 🎯 That’s exactly what that loop should print.

Let’s line up what you’ve mastered so far:

Classic for structure: for (start; condition; update) { ... }
Using an index to read array values: nums[i]
for...of to go over values directly (Python-style)
Last key idea for this lesson: using console.log to trace execution inside different kinds of loops.

I want you to compare classic for and for...of side by side.

In your editor, write and run this full program:

Javascript
```
let nums = [5, 6, 7];

console.log("---- classic for ----");
for (let i = 0; i < nums.length; i = i + 1) {
  console.log("i:", i, "nums[i]:", nums[i]);
```

```
}

console.log("---- for...of ----");
for (const n of nums) {
  console.log("n:", n);
}
Hit Run to check it, then Submit code.

After you see the output, tell me: in your own words, when would you choose classic for instead of **for...of?

Javascript
let nums = [5, 6, 7];

console.log("---- classic for ----");
for (let i = 0; i < nums.length; i = i + 1) {
  console.log("i:", i, "nums[i]:", nums[i]);
}

console.log("---- for...of ----");
for (const n of nums) {
  console.log("n:", n);
}
Exactly the output I'd expect 💪 Nice.


Here's a simple way to decide:


Use for...of when you only care about the values (5, 6, 7)
Use classic for when you also need the index (0, 1, 2) or very precise control over the counter

So for example, if you wanted to print only values at even indexes, classic for is helpful because you can use i and check i % 2 === 0.

Quick check-in for you, step by step:


Imagine this goal: "Print only the values at even positions (index 0, 2, 4, …)."


Which loop fits better for that job?
```

A classic for (let i = 0; i < arr.length; i = i + 1) ...
Or for (const value of arr) ...

You can just answer: classic for or for...of.

the classic

Exactly 💯

To check “even positions”, you need the index, so classic for is the better fit.

So you now have the three big pieces for this lesson:

Classic for (start; condition; update) → when you need an index / counter
for...of → when you just want each value, Python-style
Using console.log inside loops to trace what’s happening
Last tiny step: I want to hear your summary.

In one short sentence:

When would you choose for...of instead of a classic for loop?

id use for of to simply print out the values, and a classic when i need precice control like ony working with evens or printing an ejnumerated list it 1: value , 2: value

That’s a great summary, Ron 👏

You’ve got the exact idea: for...of for simple value iteration, classic for when you need indexes or precise control.

You’ve effectively hit all the goals for this lesson:

You understand the classic for structure and when to use it.
You can use for...of like Python’s for item in items.
You used console.log to trace what’s happening inside loops.
You’ve mastered what you need here 🎯

Do you feel ready to wrap up this For loops lesson and move

on to the next one, or do you want to play with one more loop example first?

Love it. Let’s lock this in 🔒

You now know:

How a classic for (start; condition; update) loop works, and when you want that index/control.

How for...of is the clean choice when you just want values (like Python’s for item in items).

How to drop console.log inside loops to trace indexes, values, and order of execution.

As you go forward, any time you write a loop, pause and ask:

“Do I need the index?” → classic for

“Do I just need each value?” → for...of

You’re ready for the next lesson 🚀

When you’re set, hit Next lesson in the Learn tab and I’ll be right there with you.

How would you rate this lesson?

```
let nums = [5, 6, 7];

console.log("---- classic for ----");
for (let i = 0; i < nums.length; i = i + 1) {
  console.log("i:", i, "nums[i]:", nums[i]);
}

console.log("---- for...of ----");
for (const n of nums) {
  console.log("n:", n);
}
```